

Indian Institute of Technology Indore

Line Follower Robot Report

MAKERSPACE, IC152



Batch B1, Group 6

Name	Roll number
KRISH PRASAD	240002031
NINAD KULKARNI	240002032
MADDI HARSHINI	240002033
MAMINDLAPELly SAIKRISHNA	240002034
MANAGARI SAATVIK	240002035

1. Introduction

This report details the design, development, and implementation of a line follower robot utilizing an ESP32 microcontroller board with 38 pins, infrared (IR) sensors, BO motors, a custom-designed 3D-printed chassis with wheels, and an additional gas sensor for environmental monitoring. The robot autonomously follows a predefined path marked by a line, maps the traversed path using software, and detects gas concentration levels, flagging locations where gas levels exceed a predefined threshold. This project integrates hardware design, sensor interfacing, and software development to achieve a functional robotic system with practical applications.

2. Component Description

2.1 ESP32 Microcontroller (38-pin)

The ESP32 is a powerful, dual-core microcontroller with built-in Wi-Fi and Bluetooth capabilities, making it ideal for IoT-based robotic applications. The 38-pin variant provides ample GPIO pins for interfacing multiple sensors and actuators. In this project, the ESP32 serves as the central processing unit, handling:

- Signal processing from the IR sensor for line detection.
- Analog data acquisition from the gas sensor.
- PWM signal generation for BO motor control.
- Path mapping and data logging for software visualization.



2.2 Infrared (IR) Sensor

The IR sensor module consists of an IR transmitter and receiver pair, used to detect the contrast between a black line and a white surface. The sensor outputs a digital signal based on the reflected IR light, enabling the robot to differentiate between the line and the background. Multiple IR sensors are mounted on the underside of the chassis to ensure accurate line tracking, with the ESP32 processing these inputs to adjust motor speeds for smooth navigation.



2.3 BO Motors

BO (Battery Operated) motors are compact DC motors with high torque, suitable for driving the robot's wheels. Each motor is paired with a wheel and controlled via a motor driver interfaced with the ESP32. The motors operate in differential drive mode, where varying speeds between the left and right motors allow the robot to follow curves and turns along the line path.



2.4 Custom-Designed Chassis and Wheels

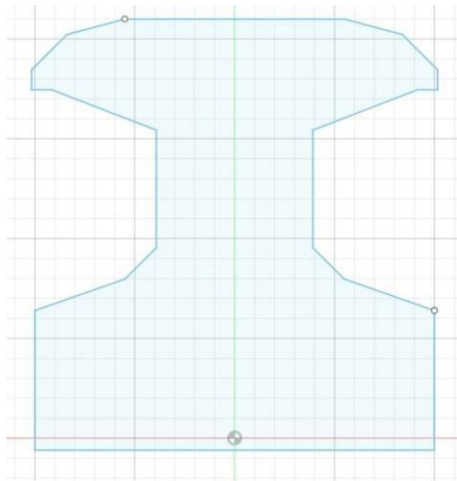
The chassis and wheels were designed using CAD software (e.g., Fusion 360) and 3D-printed using PLA filament for lightweight and durable construction. These components were tailored to optimize the robot's functionality, stability, and aesthetics.

2.4.1 Chassis

The chassis serves as the structural backbone of the robot, housing the ESP32, motor driver, sensors, battery pack, and other components. Key design features include:

- **Modular Layout:** The chassis has designated mounting points for IR sensors at the front bottom, ensuring optimal line detection, and slots for the gas sensor to maximize exposure to the environment.

- **Design:** The chassis dimensions were optimized to minimize weight while maintaining structural integrity, with a low center of gravity to enhance Compact stability during movement.
- **Cable Management:** Integrated channels and clips were designed to organize wiring, reducing the risk of loose connections and improving the robot's appearance.
- **Material Choice:** PLA filament was selected for its ease of printing, cost-effectiveness, and sufficient strength for prototyping. The chassis was printed with a layer height of 0.2 mm to balance surface finish and print time.



2.4.2 Wheels

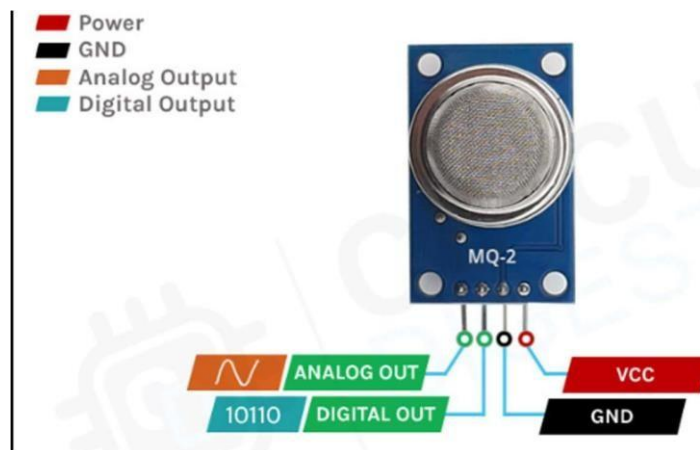
The wheels were custom-designed to complement the BO motors and ensure effective traction. Key aspects of the wheel design include:

- **Tread Pattern:** The wheels feature a textured tread pattern to provide grip on various surfaces, such as smooth floors or slightly uneven terrains, preventing slippage during sharp turns.
- **Size and Fit:** The wheel diameter and hub were designed to match the BO motor shafts, ensuring a secure fit without additional adapters. The diameter was chosen to balance speed and torque for efficient line following.
- **Material and Printing:** Like the chassis, the wheels were 3D-printed using PLA filament. A higher infill percentage (e.g., 30%) was used to ensure durability under the robot's weight and dynamic loads.
- **Alignment:** The wheels were designed with precise alignment to the chassis, ensuring smooth differential drive operation and minimizing wobbling.



2.5 Gas Sensor

A key feature of this robot is the inclusion of an analog gas sensor (e.g., MQ-series) to detect the presence of gases such as carbon monoxide, methane, or LPG. The sensor outputs an analog voltage proportional to the gas concentration, which is read by the ESP32's ADC (Analog-to-Digital Converter) pins. The system is programmed to flag locations where gas levels exceed a predefined threshold, adding an environmental monitoring capability to the robot.



3. System Architecture

The robot's architecture integrates hardware and software components to achieve line following, gas detection, and path mapping. The ESP32 processes inputs from the IR and gas sensors, controls the BO motors via a motor driver (e.g., L298N), and communicates with a software interface for path visualization. The system operates as follows:

1. **Line Following:** IR sensors detect the line, and the ESP32 adjusts motor speeds using a PID (Proportional-Integral-Derivative) control algorithm to keep the robot on track.
2. **Gas Detection:** The gas sensor continuously monitors gas levels, with the ESP32 logging data and flagging high-concentration zones.
3. **Path Mapping:** The robot's movement is tracked using odometry (based on motor rotations) and logged for visualization in software, with gas concentration data overlaid to highlight hazardous areas.

4. Software Implementation

The software is developed using the Arduino IDE with ESP32 support, leveraging libraries for sensor interfacing and motor control. Key functionalities include:

- **Line Following Algorithm:** A PID controller processes IR sensor data to compute motor speed adjustments, ensuring smooth and accurate tracking.
- **Gas Sensor Calibration:** The gas sensor is calibrated to map analog voltages to gas concentration levels, with a threshold defined for alerts.
- **Path Mapping:** A custom software interface (e.g., developed in Python or Processing) visualizes the robot's path and marks locations where gas levels exceed the threshold. Data is transmitted from the ESP32 via serial communication or Wi-Fi.

CODE:

```
// IR sensor pins
const int irFrontLeft = 36; // Front-left IR sensor
const int irFrontCenter = 35; // Front-center IR sensor
const int irFrontRight = 34; // Front-right IR sensor

// Motor A (right)
const int motorPin1 = 12; // Right motor IN1
const int motorPin2 = 27; // Right motor IN2
const int ena = 25; // Right motor PWM (enable pin for speed control)
const int gasLedpin = 32;
const int gassensorpin = 23;
// Motor B (left)
const int motorPin3 = 14; // Left motor IN3
const int motorPin4 = 33; // Left motor IN4
const int enb = 26; // Left motor PWM (enable pin for speed control)

// Motor speed settings (0-255 for PWM)
const int baseSpeed = 160; // Base speed for forward movement
const int turnSpeed = 160; // Speed during pivot turns
const int maxSpeed = 160; // Maximum forward speed
const int reverseSpeed = 160; // Reverse speed
const int gasThreshold = 600;

void setup()
{
  Serial.begin(9600); // Initialize serial communication
  randomSeed(analogRead(0)); // Seed random generator

  // Set motor pins
  pinMode(motorPin1, OUTPUT);
  pinMode(motorPin2, OUTPUT);
  pinMode(motorPin3, OUTPUT);
  pinMode(motorPin4, OUTPUT);
  pinMode(ena, OUTPUT);
  pinMode(enb, OUTPUT);

  // Set sensor pins
  pinMode(irFrontLeft, INPUT);
  pinMode(irFrontCenter, INPUT);
  pinMode(irFrontRight, INPUT);
}

void loop()
{
  int frontLeft = digitalRead(irFrontLeft);
  int frontCenter = digitalRead(irFrontCenter);
  int frontRight = digitalRead(irFrontRight);

  Serial.print("FL ");
  Serial.print(frontLeft);
```

```
Serial.print(" FC ");
Serial.print(frontCenter);
Serial.print(" FR ");
Serial.println(frontRight);

// Check gas levels
int gasValue = digitalRead(gassensorpin);
Serial.print("Gas value: ");
Serial.println(gasValue);

if (gasValue == 1)
{
  digitalWrite(gasLedpin, HIGH); // Gas detected - turn on LED
  Serial.println("Gas threshold exceeded!");
  Serial.print("Gas Value: ");
  Serial.println(gasValue);
  stopMotors();
}
else
{
  digitalWrite(gasLedpin, LOW); // Safe - turn off LED
}

navigate(frontLeft, frontCenter, frontRight);

delay(10); // Small delay for stability
}

void navigate(int fl, int fc, int fr)
{
  if ((fl == HIGH && fr == HIGH) || (fl == HIGH && fc == HIGH && fr == HIGH))
  {
    Serial.println("Dead end or junction detected - Moving backward");
    turnSharpRight();
    return;
  }

  if (fc == HIGH && fl == LOW && fr == LOW)
  {
    Serial.println("Moving Forward (centered)");
    moveForward(maxSpeed);
  }
  else if (fl == HIGH && fc == LOW && fr == LOW)
  {
    Serial.println("Sharp Turn Left");
    stopMotors();
    turnSharpLeft();
  }
  else if (fr == HIGH && fc == LOW && fl == LOW)
  {
    Serial.println("Sharp Turn Right");
    stopMotors();
    turnSharpRight();
  }
  else if (fc == HIGH && fl == HIGH && fr == LOW)
  {
    Serial.println("Slight Left");
    stopMotors();
    turnLeft();
  }
  else if (fc == HIGH && fl == LOW && fr == HIGH)
```

```

else if (fc == HIGH && fl == LOW && fr == HIGH)
{
    Serial.println("Slight Right");
    stopMotors();
    turnRight();
}
else if (fl == LOW && fc == LOW && fr == LOW)
{
    Serial.println("Lost - Move Backward");
    moveForward(reverseSpeed);
}
else
{
    Serial.println("Unclear State - Move Backward");
    moveForward(reverseSpeed);
}
}

// Motor control functions
void moveForward(int speed)
{
    digitalWrite(motorPin1, HIGH);
    digitalWrite(motorPin2, LOW);
    digitalWrite(motorPin3, HIGH);
    digitalWrite(motorPin4, LOW);
    analogWrite(ena, speed);
    analogWrite(enb, speed);
}

void moveBackward(int speed)
{
    digitalWrite(motorPin1, LOW);
    digitalWrite(motorPin2, HIGH);
    digitalWrite(motorPin3, LOW);
    digitalWrite(motorPin4, HIGH);
    analogWrite(ena, speed);
    analogWrite(enb, speed);
}

void turnLeft()
{
    digitalWrite(motorPin1, LOW); // Right motor backward
    digitalWrite(motorPin2, HIGH);
    digitalWrite(motorPin3, HIGH); // Left motor forward
    digitalWrite(motorPin4, LOW);
    analogWrite(ena, 200);
    analogWrite(enb, 200);
}

void turnRight()
{
    digitalWrite(motorPin1, HIGH); // Right motor forward
    digitalWrite(motorPin2, LOW);
    digitalWrite(motorPin3, LOW); // Left motor backward
    digitalWrite(motorPin4, HIGH);
    analogWrite(ena, 200);
    analogWrite(enb, 200);
}

```

```

void stopMotors()
{
    digitalWrite(motorPin1, LOW); // Right motor backward
    digitalWrite(motorPin2, HIGH);
    digitalWrite(motorPin3, HIGH); // Left motor forward
    digitalWrite(motorPin4, LOW);
    analogWrite(ena, 0);
    analogWrite(enb, 0);
    delay(20);
}

void turnSharpRight()
{
    digitalWrite(motorPin1, HIGH); // Right motor forward
    digitalWrite(motorPin2, LOW);
    digitalWrite(motorPin3, LOW); // Left motor backward
    digitalWrite(motorPin4, HIGH);
    analogWrite(ena, 255);
    analogWrite(enb, 255);
}

void turnSharpLeft()
{
    digitalWrite(motorPin1, LOW); // Right motor backward
    digitalWrite(motorPin2, HIGH);
    digitalWrite(motorPin3, HIGH); // Left motor forward
    digitalWrite(motorPin4, LOW);
    analogWrite(ena, 255);
    analogWrite(enb, 255);
}

```

5. Explanation of the code

- The code controls a line-following robot that can detect gas and respond accordingly.
- It uses three IR sensors (left, center, right) to follow a black line on the ground.
- Based on sensor input, the robot decides whether to move forward, take slight turns, or take sharp 90-degree turns.
- Sharp turns (left or right) are executed when only one side IR sensor detects the line, allowing the robot to make precise 90-degree corner turns on sharp bends or T-junctions.
- If all sensors detect the line (dead end or junction), the robot performs a sharp right turn to recover.
- If no sensor detects the line, it moves backward to re-align.
- A gas sensor is integrated to monitor the environment. If gas is detected, the robot stops and alerts via LED.
- Motor speed is controlled via PWM, and movement decisions are made in real time using sensor feedback

6. Additional Design Justifications

Why did we use 3 IR sensors instead of 2?

- Three sensors provide better accuracy in detecting the line's position, enabling the robot to handle curves, junctions, and sharp turns effectively.
- Two sensors would struggle to distinguish between minor deviations and sharp turns, often resulting in poor navigation.

Why front-wheel sensor placement?

- Sensors are placed near the front so the robot can detect changes in the path ahead of time, allowing it to act before reaching a turn.
- Analogy: Controlling a long, thin stick along a random path is easier when holding it from the front rather than the back—this gives better control and quicker response.

Why did it fail on a smooth surface?

- On smooth surfaces, the robot failed to perform sharp turns correctly due to the lack of friction.
- When trying to stop and turn, it kept sliding forward because of momentum, causing it to overshoot turns and lose alignment with the path.

ff

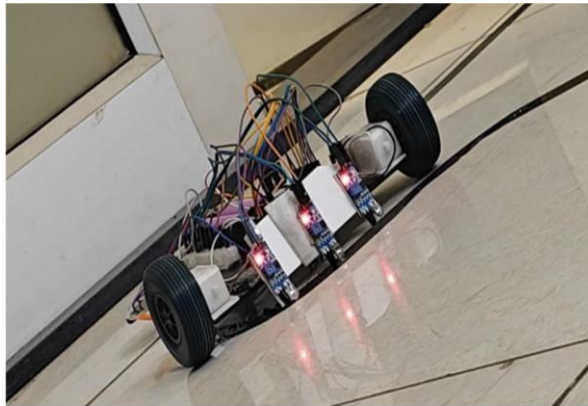
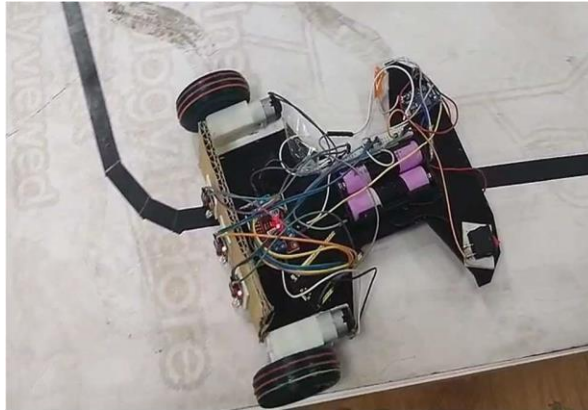
7. Unique Features

7.1 Gas Detection and Threshold Alerts

The integration of a gas sensor adds a novel environmental monitoring capability. The robot not only follows a line but also identifies areas with potentially hazardous gas concentrations, making it suitable for applications like industrial safety or environmental surveillance.

7.2 Custom 3D-Printed Design

The 3D-printed chassis and wheels are tailored to the robot's requirements, optimizing weight, sensor placement, and aesthetics. The modular chassis and grippy wheels demonstrate the use of additive manufacturing in robotics prototyping.



7.3 Path Mapping with Gas Data Visualization

The software interface provides a real-time map of the robot's path, overlaid with gas concentration data. This feature enhances the robot's utility in scenarios requiring spatial analysis of environmental conditions.

8. Learnings

Throughout the project, the team gained valuable insights, including:

- **Hardware Interfacing:** Understanding the ESP32's GPIO and ADC capabilities for sensor and motor integration.
- **3D Printing:** Designing and iterating on chassis and wheel designs to balance functionality, weight, and structural integrity.
- **PID Tuning:** Fine-tuning PID parameters for optimal line-following performance, addressing issues like overshooting and oscillation.
- **Sensor Calibration:** Calibrating the gas sensor to account for environmental variables and ensure accurate readings.
- **Team Collaboration:** Coordinating tasks across hardware design, software development, and testing to meet project goals.

Challenges included aligning IR sensors for consistent line detection, mitigating noise in gas sensor readings, and optimizing 3D print settings for durability, which were addressed through iterative testing and software filtering.

9. **Future Scope**

The project has significant potential for further development, including:

- **Wireless Data Transmission:** Implementing Wi-Fi-based data streaming to a cloud server for real-time monitoring.
- **Advanced Path Planning:** Integrating SLAM (Simultaneous Localization and Mapping) for dynamic path generation in unknown environments.
- **Multi-Sensor Integration:** Adding sensors like temperature, humidity, or ultrasonic for enhanced environmental awareness.
- **Autonomous Navigation:** Upgrading the robot to avoid obstacles using additional sensors and algorithms.
- **Energy Efficiency:** Optimizing power consumption through low-power modes and efficient motor control.

10. **Conclusion**

The line follower robot, built using an ESP32, IR sensors, BO motors, a custom 3D-printed chassis and wheels, and a gas sensor, successfully demonstrates autonomous navigation and environmental monitoring. The integration of gas detection and path mapping, along with a tailored chassis and wheel design, sets this project apart, showcasing its potential in real-world applications like industrial monitoring and safety inspections. The project provided hands-on experience in robotics, sensor interfacing, 3D printing, and software development, laying a strong foundation for future innovations in autonomous systems.

9. **Drive link of working:**

<https://drive.google.com/drive/folders/1zZ2FmnAQqspX9rY5HLva-MoNlyg4FBCa?usp=sharing>